

PRÜFUNGEN REPRODUZIERBAR IN MOODLE-XML ÜBERFÜHREN

11. JUNI 2026

Thomas Lampart | [✉ thomas.lampart@students.ffhs.ch](mailto:thomas.lampart@students.ffhs.ch)
github.com/dlh-ki-xml

Prüfungen lokal, prüfbar und reproduzierbar in Moodle-XML überführen.



WORUM GEHT ES?

- Schulen investieren in digitale Lernumgebungen wie Moodle.
- Viele Lehrpersonen nutzen diese Systeme noch zurückhaltend.
- Lehrfreiheit und Zeitdruck erschweren den Wechsel zu digitalen Prüfungsprozessen.
- Für Lernende und Institutionen entsteht dadurch ein uneinheitlicher digitaler Prüfungsalltag.
- Das Projekt senkt die Einstiegshürde: bestehende Prüfungen werden kontrolliert nach Moodle überführt.

WAS WÄRE ANZUSTREBEN?

FÜR LERNENDE

- zeitgemässe Prüfungskultur
- klarere Aufgabenformate
- weniger Missverständnisse bei der Bearbeitung von Aufgaben
- schnellere Rückmeldungen bei geeigneten Fragetypen

FÜR LEHRPERSONEN UND INSTITUTIONEN

- niederschwelliger Einstieg ins digitale Prüfen
- mehr LMS-Nutzung durch konkreten Alltagsnutzen
- nachvollziehbare und wiederholbare Prüfungsprozesse

Zielbild

Digitale Prüfungen sollen verständlicher, machbarer und institutionell skalierbar werden.

WAS SOLL DER GANZ KONKRETE NUTZEN FÜR LEHRPERSONEN SEIN?

DIREKTER NUTZEN

- bestehende Prüfungen einfach digitalisieren, statt umständlich zu erfassen
- Moodle-XML nutzen, ohne XML-Details kennen zu müssen
- Unsicherheiten und didaktische Kontrolle über Review-Hinweise selber steuern (Teacher in the Loop)
- RAG: relevante Informationen (Skripte, Lernziele) gezielt einbeziehen

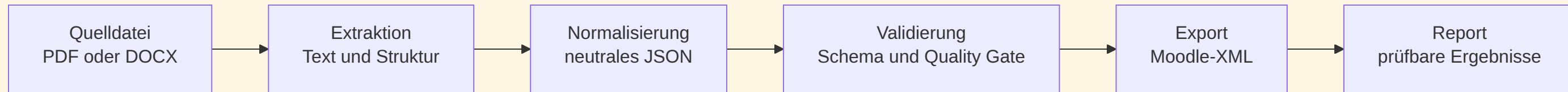
NUDGE UND ANSCHLUSSFÄHIGKEIT

- vorhandene Prüfungen werden zum Einstiegspunkt
- der Nutzen entsteht sofort im Arbeitsalltag
- JSON dient als prüfbarer Stabilitätsanker
- in weitere Formate konvertierbar, z. B. für ILIAS

Das Projekt senkt die Einstiegshürde: Aus vorhandenen Prüfungen entstehen schneller didaktisierte digitale Prüfungen.

VORGEHEN

WIE WIRD DAS METHODISCH ERREICHT?



TECHNISCHES PRINZIP

- keine direkte Blackbox-Konvertierung
- jede Stufe erzeugt prüfbare Zwischenergebnisse
- Quality Gate markiert Unsicherheiten vor dem Export

KERNIDEE

Die Pipeline trennt Extraktion, Interpretation, Prüfung und Export. Dadurch bleibt der Prozess nachvollziehbar und korrigierbar.

WARUM IST PDF/DOCX-PARSING ANSPRUCHSVOLL?

DIE DATEI IST NICHT DIE BEDEUTUNG

- Layout, Farben und Abstände tragen fachliche Information
- Lösungen können visuell markiert statt textlich ausgezeichnet sein
- Tabellen und Bildaufgaben verlieren beim reinen Textauszug Struktur
- derselbe Inhalt kann mehrere sinnvolle Moodle-Fragetypen ergeben

1) Dieses Bild trägt den Titel "Selbstbildnis mit Judenpass". Es wurde vom Maler Felix Nussbaum erstellt. ¶



1) Aufgrund der Merkmale auf dem Bild lässt sich das Datum der Entstehung dieses Selbstbildnisses vermuten. Nennen Sie. ¶

a) Welche Merkmale sind mit welchem Jahr verbunden? /1 ¶
Das J im Pass war ab 1938 obligatorisch, das Tragen des Sterns auf der Jacke ab 1941. ¶

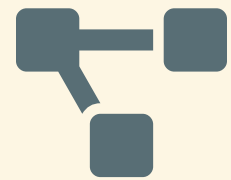
b) Wann wurde dieses Bild frühestens gemalt? /1 ¶

Das Bild muss deshalb nach 1941 erstellt worden sein. ¶

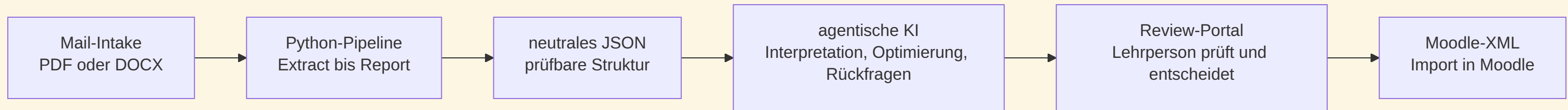
1b) Mutmassen Sie: Wie ist der Maler mit hoher Wahrscheinlichkeit gestorben und wo? → /2 ¶

Er starb im Vernichtungslager Auschwitz, man nahm lange an, er sei vergast worden, es ist aber unklar, er könnte auch bei der Arbeit gestorben sein. ¶

Lösungssignale über Farbe statt Struktur



EINBETTUNG: PYTHON-PIPELINE ALS KERNBAUSTEIN



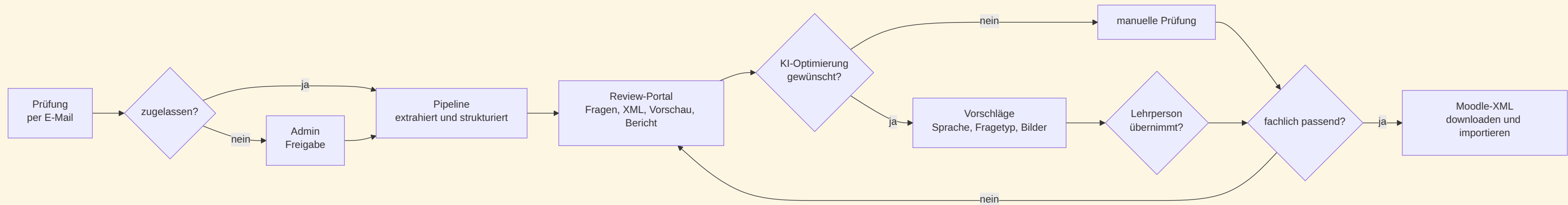
HEUTE OPTIMIERT

- lokaler Python-Workflow
- neutrales JSON-Zwischenformat
- Validierung, Quality Gate und Moodle-XML-Export

SPÄTER ERGÄNZT

- E-Mail-Intake und Antwortpfad
- agentische KI-Loops für Interpretation und Optimierung
- Review-Portal mit Freigabe durch die Lehrperson

ZIELZUSTAND: KONTROLLIERTER REVIEW-PROZESS

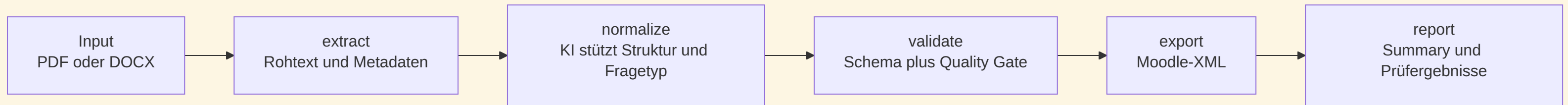


Prinzip

Die Pipeline automatisiert den Transfer. Die KI schlägt Verbesserungen vor. Die Lehrperson entscheidet.

RESULTATE

RESULTAT 1: PRÜFBARER TRANSFORMATIONSKERN



WAS STEHT HEUTE?

- lokale Python-Pipeline mit festen Stages
- neutrales JSON als prüfbares Zwischenformat
- Moodle-XML-Export mit Report pro Lauf

WARUM IST DAS BELASTBAR?

- jeder Schritt bleibt lokal nachvollziehbar
- Quality Gate stoppt oder markiert unsichere Fälle
- Export bleibt deterministisch statt KI-Direktausgabe

RESULTAT 2: AN DREI PRÜFUNGEN KALIBRIERT

Prüfung	Fokus	Ergebnis
Geschichtsprüfung 1	Frage typ, Punkte, Segmentierung	Phase-1-Eval: total_score = 1.0000
Geschichtsprüfung 2	Generalisierung auf zweite reale Prüfung	Struktur und Typverteilung entsprechen Referenz
Deutschprüfung	AUF/LSG-Logik und PDF-/DOCX-Verarbeitung	Aufgabenfassung und Lösungshinweise trennbar

VERBESSERT

- Confidence-Logik stabilisiert
- AUF und LSG sauberer getrennt
- PDF-Header, Fusszeilen und Trennsilben bereinigt

EINORDNUNG

- keine ML-Trainingsphase, sondern regel- und promptbasierte Kalibrierung
- reale Prüfungen statt synthetischer Beispiele
- offene Deltas bleiben im Report sichtbar

RESULTAT 3: VERGLEICH GEGEN KI-DIREKTEXPORT

0.945

Pipeline-Score gegen Referenz-XML

0.636 / 0.458

Copilot-Studio Run 1 / Run 2

0/2

strikt gültige Copilot-XMLs im Test

MESSPUNKT

- Referenzvergleich für Geschichtsprüfung
- bewertet: XML-Validität, Fragen, Typen, Punkte und Textabdeckung
- Report: `xml-reference-comparison.md`

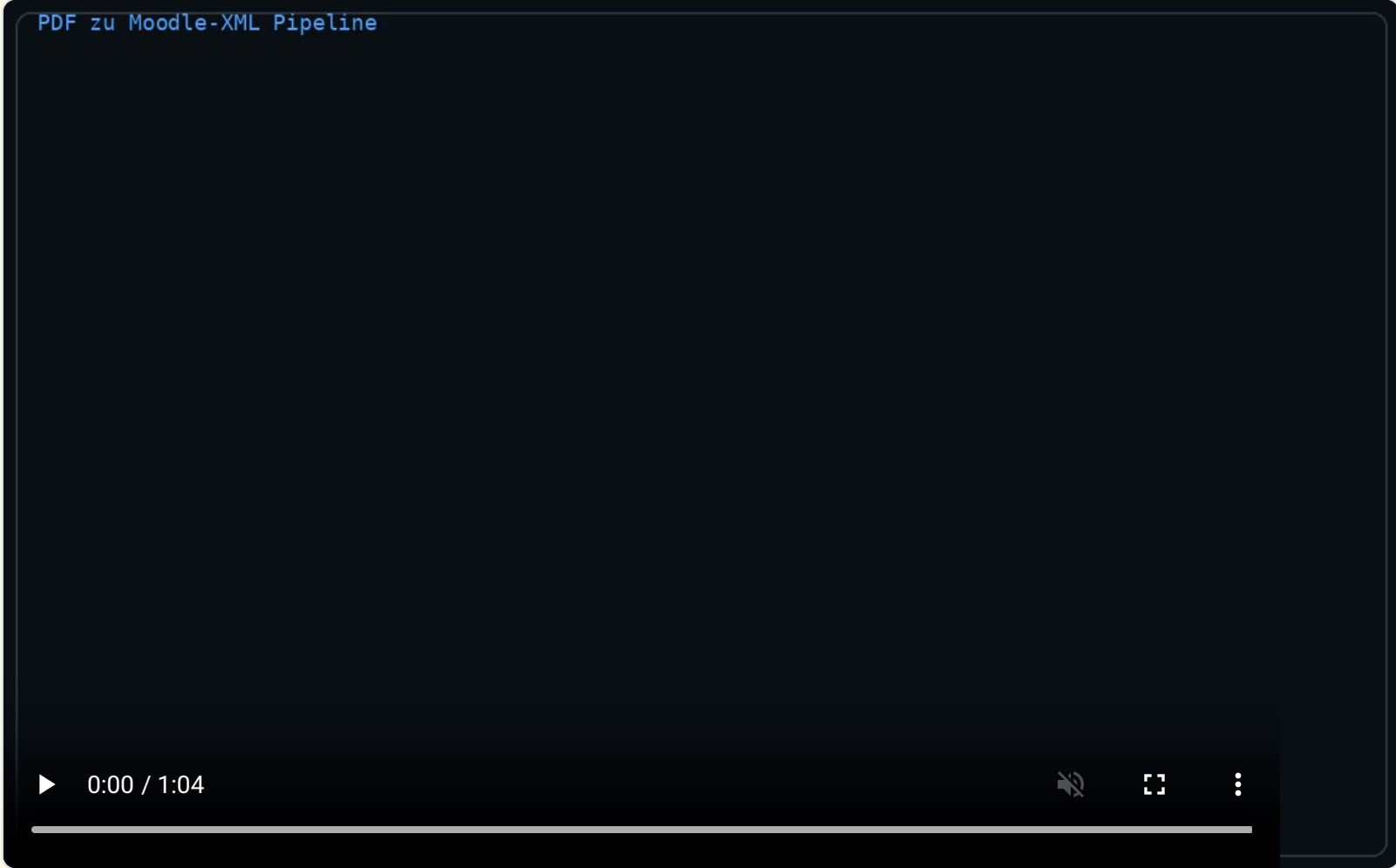
AUSSAGE

- Pipeline ist näher an der Referenz
- Copilot-Runs unterscheiden sich deutlich voneinander
- deterministischer Export ist konsistenter und import-sicherer

>_ DEMO UND AUSBLICK

DEMO-PFAD: REPRODUZIERBARER AUSFÜHRUNGSPFAD

PIPELINE-LAUF



PRÜFDATEIEN

Prompt	Struktur, Fragetyp, Revi
01_extract/raw_text.txt	Rohtext
02_normalize/normalized.json	Aufgabenstruktur
03_validate/validation_report	Quality Gate
05_report/summary.md	Kurzstatus

ZENTRALE LIBRARIES

PyMuPDF	PDF-Extraktion
python-docx	DOCX-Extraktion
pydantic	Validierung
lxml	Moodle-XML
pytest	Tests

Einordnung

Das Video zeigt, wie eine PDF-Prüfung durch die Pipeline läuft und welche Dateien den Lauf nachvollziehbar machen.

AUSBLICK: PYTHON-BACKBONE MIT KI-WORKFLOW-LOGIK

BACKBONE VON MIR

- operativer Backbone
- lokale Tests und deterministische Prüfergebnisse
- Nachvollziehbarkeit pro Stage

WORKFLOW VON MARTIN

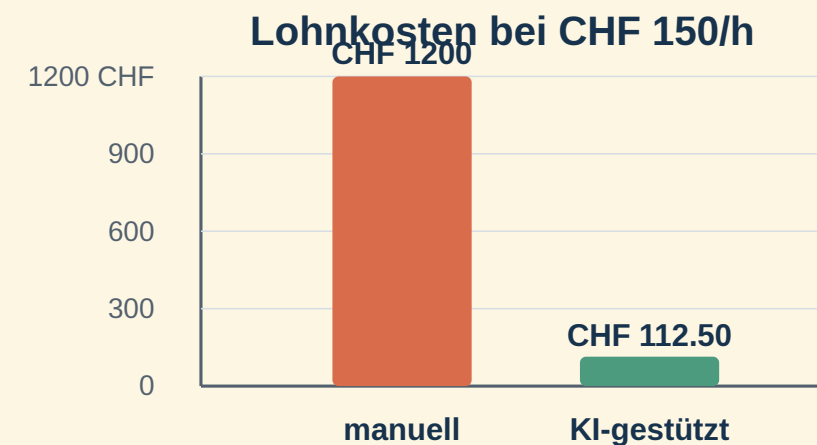
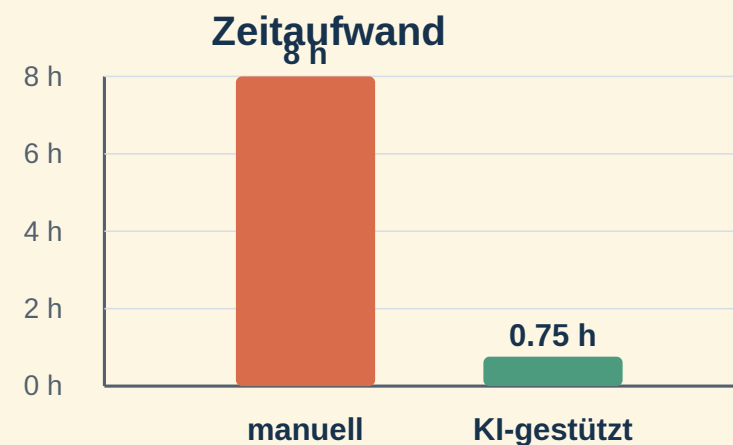
- E-Mail-Intake und Rückfragepfad
- agentische KI-Loops für Interpretation, didaktische Optimierung und Rückfragen
- neutrales Zwischenformat (JSON) und Entscheidungslogik

Strategische Richtung

Python-Pipeline als Ausführungs-Backbone behalten und die fachliche Qualitätslogik des KI-Workflows schrittweise integrieren.



- Prüfungen einfach digitalisieren und didaktisch optimieren statt umständlich manuell zu erfassen
- robuste und deterministische Python-Pipeline mit prüfbaren Stages als technischer Backbone
- KI-gestützte Interpretation, Qualitätsprüfung und Optimierung mit «Teacher in the Loop»
- 94.5 % Übereinstimmung mit der Referenz-XML in der Geschichtsprüfung; Grammatikprüfungen als Aufgaben- und Lösungsversion in DOCX/PDF erfolgreich verarbeitet
- Nächster Schritt: agentischer KI-Workflow systematisch aufbauen und in die Pipeline integrieren



DANKE FÜR EURE AUFMERKSAMKEIT!

Ich freue mich über Kritik und Anregungen!



[✉ thomas.lampart@students.ffhs.ch](mailto:thomas.lampart@students.ffhs.ch)



[🔗 slides](#)
[project-slides](#)
[project-docs](#)

[🔗 Workflow: dlh-ki-xml@svc-ai.fyi](#)



[🔗 repositories](#)

ANHANG: INHALTSVERZEICHNIS

REPRODUKTION

- A: Repo-Struktur für Wiederholung
- B: Minimaler Reproduktionsablauf
- C: Was wurde iterativ verbessert?
- D: Belegte Run-Ergebnisse
- E: Zielarchitektur der Kombination

HINTERGRUND ZUR PIPELINE

- F: Module und Verantwortlichkeiten
- G: Zwischenformat und Quality Gate
- H: Python-Libraries
- I: Evaluation und Vergleichsreports
- J: Vibecoding-Workflow

REPRODUKTION

ANHANG A: REPO-STRUKTUR FÜR WIEDERHOLUNG

```
project-docs/  
├── project/python-skripte/  
│   ├── data/input_pdfs/  
│   ├── data/runs/<exam-id>/  
│   ├── src/pdf_to_moodle/  
│   └── tests/  
└── results/  
  
project-slides/  
└── GenKI2-SemA-final/
```

- `src/pdf_to_moodle/` enthält Extraktion, Normalisierung, Validierung und Export.
- `data/runs/` enthält alle Artefakte eines konkreten Laufs.
- `project-slides/` enthält die versionierte Präsentationsdokumentation.

[Zurück zur Anhangsübersicht](#)

ANHANG B: MINIMALER REPRODUKTIONSABLAUF

```
cd project-docs/project/python-skripte
source .venv/bin/activate

python -m pytest tests/test_parser.py tests/test_prompt_solution_split.py tests/test_quality_gate.py -q

python -m pdf_to_moodle.main \
  --exam-id deutsch-auf-p16-2026w22 \
  --input data/input_pdfs/AUF-Grammatikpruefung-Syntax-Interpunktion.docx \
  --stage all

python -m pdf_to_moodle.main \
  --exam-id deutsch-auf-p17-pdf-2026w22 \
  --input data/input_pdfs/AUF-Grammatikpruefung-Syntax-Interpunktion.pdf \
  --stage all
```

Erwartung: In beiden Runs enthält `05_report/summary.md` den Status `quality_gate: ready`.

[Zurück zur Anhangsübersicht](#)

ANHANG C: WAS WURDE ITERATIV VERBESSERT?

Iteration	Problem	Wirkung
Confidence-Fix	systematisches needs_review	History-Run auf ready gebracht
AUF vs. LSG	Aufgabenhinweise landeten in solution_text	AUF-Dateien sauberer modelliert
PDF-Cleanup	Header, Fusszeilen, Trennsilben	PDF-Run fachlich näher an DOCX gebracht
Prompt-Heuristik	Rubriken und Instruktionen wurden falsch getrennt	längere Aufgabenköpfe stabilisiert

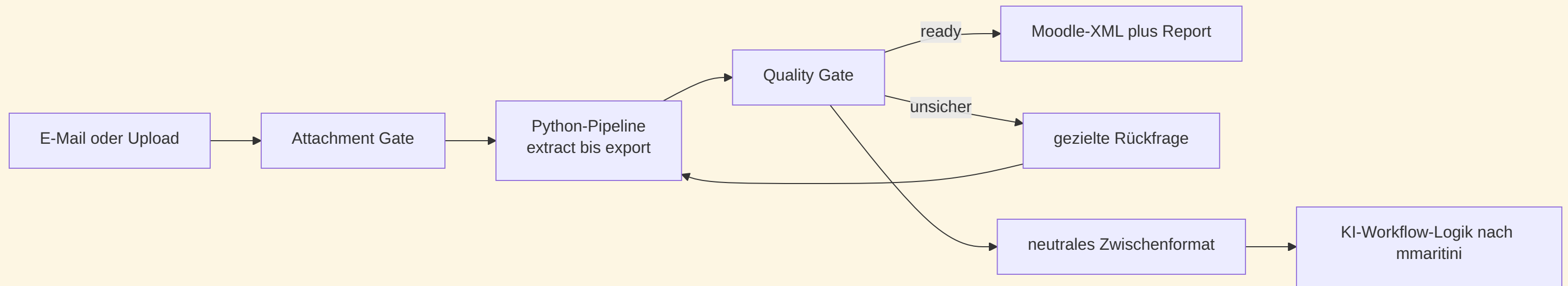
[Zurück zur Anhangsübersicht](#)

ANHANG D: BELEGTE RUN-ERGEBNISSE

Run	Ergebnisdatei	Zweck
history-p14-2026w22	05_report/summary.md	Quality Gate und Review-Status
deutsch-auf-p16-2026w22	02_normalize/normalized.json	AUF-DOCX in JSON geprüft
deutsch-auf-p17-pdf-2026w22	01_extract/raw_text.txt	PDF-Extraktion sichtbar geprüft
deutsch-auf-p17-pdf-2026w22	03_validate/validation_report.json	finale Bewertung ready

[Zurück zur Anhangsübersicht](#)

ANHANG E: ZIELARCHITEKTUR DER KOMBINATION



- technische Ausführung bleibt deterministisch
- fachliche Entscheidungslogik und Nutzerfluss werden ausgebaut
- beide Stränge zählen auf denselben Qualitätsstandard ein

[Zurück zur Anhangsübersicht](#)

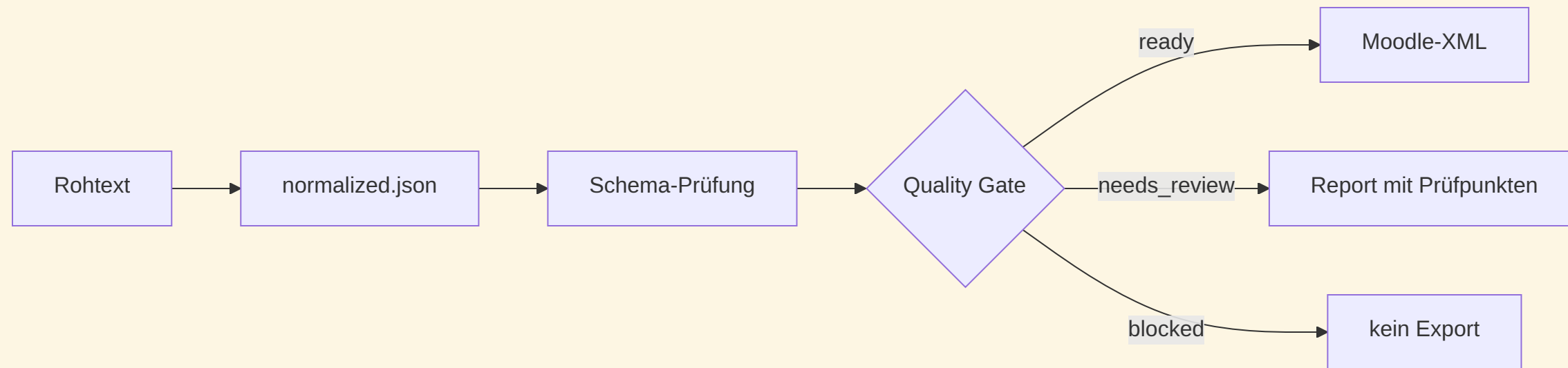
HINTERGRUND ZUR PIPELINE

ANHANG F: MODULE UND VERANTWORTLICHKEITEN

Modul	Verantwortung
pipeline.py	orchestriert extract, normalize, validate, export, report
parser.py	erkennt Aufgabenstruktur, Punkte, Antwortbereiche und Rubriken
llm_pipeline.py	modelliert KI-gestützte Normalisierung und Review-Hinweise
json_schema.py	definiert das prüfbare Zwischenformat
xml_export.py	erzeugt Moodle-XML deterministisch aus validiertem JSON
moodle_gate.py	bewertet ready, needs_review, blocked
merge_solutions.py	führt Aufgaben- und Lösungsversion konservativ zusammen

[Zurück zur Anhangsübersicht](#)

ANHANG G: ZWISCHENFORMAT UND QUALITY GATE



- `normalized.json` ist der Stabilitätsanker zwischen Parsing, KI-Logik und Export.
- Das Quality Gate verhindert, dass unsichere Fälle still als fertiger Import erscheinen.
- Review-Hinweise sind Teil des Outputs, nicht nur interne Fehlermeldungen.

[Zurück zur Anhangsübersicht](#)

ANHANG H: PYTHON-LIBRARIES

Library	Rolle in der Pipeline
PyMuPDF	PDF-Text und PDF-Strukturen extrahieren
python-docx	DOCX-Dateien, Absätze und Tabellen auslesen
pydantic	strukturierte Validierung und Fehlerkontrolle
lxml	Moodle-XML erzeugen und XML-Vergleiche auswerten
pytest	Regressionstests für Parser, Quality Gate und Reports

Prinzip
Fachliche Interpretation darf KI-gestützt sein; der Export selbst bleibt kontrolliert und deterministisch.

[Zurück zur Anhangsübersicht](#)

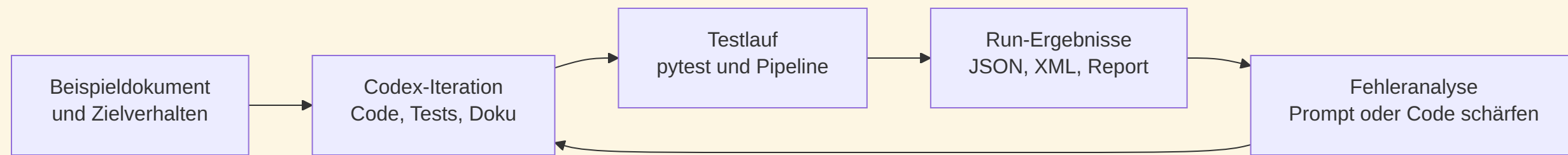
ANHANG I: EVALUATION UND VERGLEICHSREPORTS

Report	Aussage
accuracy-summary.md	verdichtet Phase-1-Cases, Total Score und Quality Gate
scoring.json	enthält die maschinenlesbaren Case-Ergebnisse
comparison-vs-ground-truth.md	zeigt Abweichungen gegen Referenz-XML
xml-reference-comparison.md	vergleicht Pipeline und Copilot-Studio gegen dieselbe Referenz

- Kernmessung: Struktur, Fragetyp, Punkte, Textabdeckung, XML-Validität.
- Die Reports trennen Korrektheit von Importfähigkeit.
- Offene Deltas bleiben sichtbar statt in einem Gesamtscore zu verschwinden.

[Zurück zur Anhangsübersicht](#)

ANHANG J: VIBECODING-WORKFLOW



- Anforderungen wurden aus realen Prüfungen und Referenz-XML abgeleitet.
- Codex unterstützte bei Implementierung, Testfällen, Reports und Dokumentation.
- Jede Verbesserung wurde über lokale Tests oder konkrete Run-Ergebnisse überprüft.

[Zurück zur Anhangsübersicht](#)